

```
timescale 1ns / 1ps
`default_nettype none
`include "parameters.vh"
```

```
module aludec (
    input wire [31:0] i_instr,
    output reg [3:0] o_alu_ctrl
);
```

```
wire [2:0] is_func3;
wire is_func7;
wire [6:0] is_opcode;
```

```
assign is_func7 = i_instr[30]; // Differentiates ADD, SUB, SRL, SRA
assign is_func3 = i_instr[14:12];
assign is_opcode = i_instr[6:0];
```

```
always @(*) begin
    case (is_opcode)
        `R: begin
            case (is_func3)
                `ADDI: o_alu_ctrl = (is_func7) ? `SUB : `ADD;
                `SRLI: o_alu_ctrl = (is_func7) ? `SRA : `SRL;
                `SLLI: o_alu_ctrl = `SLL;
                `SLTI: o_alu_ctrl = `SLT;
                `SLTUI: o_alu_ctrl = `SLTU;
                `XORI: o_alu_ctrl = `XOR;
                `ORI: o_alu_ctrl = `OR;
                `ANDI: o_alu_ctrl = `AND;
                default: o_alu_ctrl = 4'b1111; // Invalid ALU control signal
            endcase
        end

        `I: begin
            case (is_func3)
                `ADDI: o_alu_ctrl = `ADD;
                `SRLI: o_alu_ctrl = (is_func7) ? `SRA : `SRL;
                `SLLI: o_alu_ctrl = `SLL;
                `SLTI: o_alu_ctrl = `SLT;
                `SLTUI: o_alu_ctrl = `SLTU;
                `XORI: o_alu_ctrl = `XOR;
                `ORI: o_alu_ctrl = `OR;
                `ANDI: o_alu_ctrl = `AND;
                default: o_alu_ctrl = 4'b1111; // Invalid ALU control signal
            endcase
        end
    end
```

```

`LD, `S, `JR, `UPC, `J, `U: o_alu_ctrl = `ADD;

`B: begin
    case (is_func3)
        `BEQ, `BNE, `BLT, `BGE, `BLTU, `BGEU: o_alu_ctrl = `ADD;
        default: o_alu_ctrl = 4'b1111; // Invalid ALU control signal
    endcase
end

// Atomic Memory Operations (AMO)
`AMO: begin
    case (is_func3)
        `AMO_ADD: o_alu_ctrl = `AMO_ADD;
        `AMO_AND: o_alu_ctrl = `AMO_AND;
        `AMO_OR: o_alu_ctrl = `AMO_OR;
        `AMO_XOR: o_alu_ctrl = `AMO_XOR;
        `AMO_MIN: o_alu_ctrl = `AMO_MIN;
        `AMO_MAX: o_alu_ctrl = `AMO_MAX;
        `AMO_MINU: o_alu_ctrl = `AMO_MINU;
        `AMO_MAXU: o_alu_ctrl = `AMO_MAXU;
        `AMO_SWAP: o_alu_ctrl = `AMO_SWAP;
        `AMO_LR: o_alu_ctrl = `AMO_LR;
        `AMO_SC: o_alu_ctrl = `AMO_SC;
        default: o_alu_ctrl = 4'b1111; // Invalid AMO control signal
    endcase
end

    default: o_alu_ctrl = 4'b1111; // Invalid opcode
endcase
end

endmodule

```